

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO TUẦN BA
MÔN THỰC TẬP CƠ SỞ

Giảng viên hướng dẫn : Kim Ngọc Bách

Sinh viên thực hiện : Nguyễn Trung Nghĩa

Mã Sinh Viên : B23DCCN602

Hà Nội - 2026

1. Mục tiêu hướng đến

Sau khi đã hoàn thành việc chuẩn bị dataset trong tuần thứ hai, tuần thứ ba của dự án tập trung vào giai đoạn quan trọng nhất: huấn luyện và đánh giá mô hình phát hiện đối tượng. Đây là bước chuyển từ việc chuẩn bị dữ liệu sang xây dựng một mô hình học sâu có khả năng nhận diện thực tế.

Trong tuần này, các mục tiêu chính bao gồm:

- Thực hiện sanity check để kiểm tra pipeline huấn luyện mô hình YOLOv8
- Huấn luyện mô hình với số epoch nhỏ nhằm xác nhận khả năng học của mô hình
- Tiến hành huấn luyện chính thức với số epoch lớn để tối ưu hiệu suất
- Đánh giá mô hình bằng tập validation thông qua các chỉ số chuẩn
- Phân tích kết quả bằng confusion matrix và các metrics
- So sánh kết quả giữa các giai đoạn huấn luyện

Việc hoàn thành các mục tiêu này giúp xây dựng được một mô hình có độ chính xác cao và làm cơ sở cho việc triển khai ứng dụng trong các tuần tiếp theo.

2. Huấn luyện mô hình YOLOv8

2.1. Sanity Check (Kiểm tra pipeline huấn luyện)

Trước khi tiến hành huấn luyện chính thức với số lượng epoch lớn, một bước quan trọng cần thực hiện là sanity check nhằm kiểm tra toàn bộ pipeline huấn luyện của mô hình YOLOv8. Đây là bước thử nghiệm nhanh với số epoch nhỏ để đảm bảo rằng dữ liệu và mô hình có thể hoạt động chính xác trước khi tiêu tốn nhiều tài nguyên cho quá trình huấn luyện dài.

Mục đích của bước sanity check bao gồm:

- Kiểm tra dataset có được đọc đúng bởi mô hình hay không
- Xác nhận cấu trúc file và định dạng nhãn phù hợp với YOLOv8
- Đảm bảo quá trình huấn luyện không gặp lỗi (path, format, thiếu dữ liệu, ...)
- Kiểm tra xem mô hình có khả năng học từ dữ liệu hay không (loss giảm, mAP tăng)

Trong bước này, mô hình YOLOv8n pretrained được sử dụng để huấn luyện thử nghiệm với số epoch nhỏ.

Lệnh thực hiện:

```
yolo detect train data=dataset/data.yaml model=yolov8n.pt epochs=2 imgsz=416  
batch=8 name=sanity_check
```

Giải thích các tham số:

- **data=dataset/data.yaml:** chỉ định file cấu hình dataset, bao gồm đường dẫn tới các tập train, validation và test
- **model=yolov8n.pt:** sử dụng mô hình YOLOv8 phiên bản nano đã được pretrained
- **epochs=2:** số epoch nhỏ để kiểm tra nhanh quá trình huấn luyện
- **imgsz=416:** kích thước ảnh đầu vào, phù hợp với dataset đã được resize trước đó
- **batch=8:** số lượng ảnh được xử lý trong mỗi batch, phù hợp với tài nguyên GPU hiện có
- **name=sanity_check:** tên thư mục lưu kết quả huấn luyện

Sau khi thực hiện lệnh trên, hệ thống tiến hành huấn luyện mô hình trong 2 epoch và lưu kết quả tại thư mục: **runs/detect/sanity_check/**

Thư mục này bao gồm các thông tin quan trọng như:

- File trọng số mô hình (best.pt, last.pt)
- Biểu đồ loss và metrics
- Hình ảnh minh họa nhãn (labels.jpg)

Kết quả thu được:

Sau 2 epochs huấn luyện, mô hình đạt được các chỉ số cơ bản như sau:

- Precision ≈ 0.545
- Recall ≈ 0.593
- mAP50 ≈ 0.568

Ngoài ra, các giá trị loss (box_loss, cls_loss, dfl_loss) có xu hướng giảm qua từng epoch, cho thấy mô hình đã bắt đầu học được đặc trưng từ dữ liệu.

Nhận xét:

Kết quả sanity check cho thấy:

- Dataset được đọc và xử lý đúng bởi mô hình YOLOv8

- Pipeline huấn luyện hoạt động ổn định, không phát sinh lỗi
- Mô hình có khả năng học từ dữ liệu ngay từ những epoch đầu tiên

Mặc dù các chỉ số đánh giá chưa cao do số lượng epoch ít, nhưng kết quả này là hoàn toàn hợp lý và đáp ứng mục tiêu của bước kiểm tra.

Kết luận:

Bước sanity check đã xác nhận rằng toàn bộ pipeline từ dataset đến mô hình hoạt động chính xác. Do đó, có thể tiếp tục chuyển sang giai đoạn huấn luyện chính thức với số lượng epoch lớn hơn nhằm tối ưu hiệu suất của mô hình.

2.2. Debug Training - 20 epochs (Huấn luyện thử nghiệm)

Sau khi hoàn thành bước kiểm tra pipeline huấn luyện (sanity check) và xác nhận rằng mô hình YOLOv8 có thể đọc dữ liệu và thực hiện huấn luyện mà không phát sinh lỗi, bước tiếp theo là tiến hành huấn luyện thử nghiệm (debug training) nhằm đánh giá khả năng học của mô hình trên dataset.

Mục tiêu của bước huấn luyện thử nghiệm là:

- Kiểm tra xem mô hình có thực sự học được đặc trưng từ dữ liệu hay không.
- Quan sát xu hướng giảm của hàm mất mát (loss) trong quá trình huấn luyện.
- Đánh giá sơ bộ hiệu suất của mô hình thông qua các chỉ số như Precision, Recall và mAP.
- Phát hiện các vấn đề tiềm ẩn như underfitting hoặc overfitting trước khi tiến hành huấn luyện chính thức.

Trong bước này, mô hình YOLOv8n pretrained được sử dụng để fine-tune trên dataset mũ bảo hiểm với số lượng epoch nhỏ hơn so với huấn luyện chính thức.

Lệnh thực hiện:

```
yolo detect train data=dataset/data.yaml model=yolov8n.pt epochs=20 imgsz=416  
batch=8 name=helmet_debug
```

Giải thích các tham số:

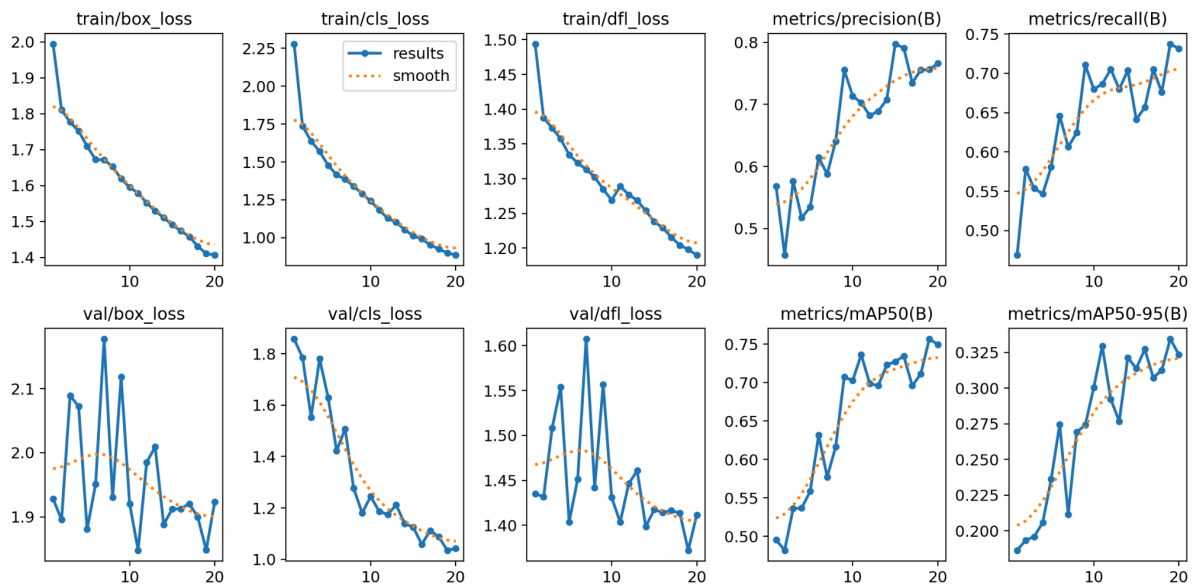
- **data=dataset/data.yaml:** chỉ định file cấu hình dataset, bao gồm đường dẫn tới các tập train, validation và test

- **model=yolov8n.pt**: sử dụng mô hình YOLOv8 phiên bản nano đã được pretrained
- **epochs=20**: số epoch huấn luyện thử nghiệm
- **imgsz=416**: kích thước ảnh đầu vào, phù hợp với dataset đã được resize trước đó
- **batch=8**: số lượng ảnh được xử lý trong mỗi batch, phù hợp với tài nguyên GPU hiện có
- **name=helmet_debug**: tên thư mục lưu kết quả huấn luyện

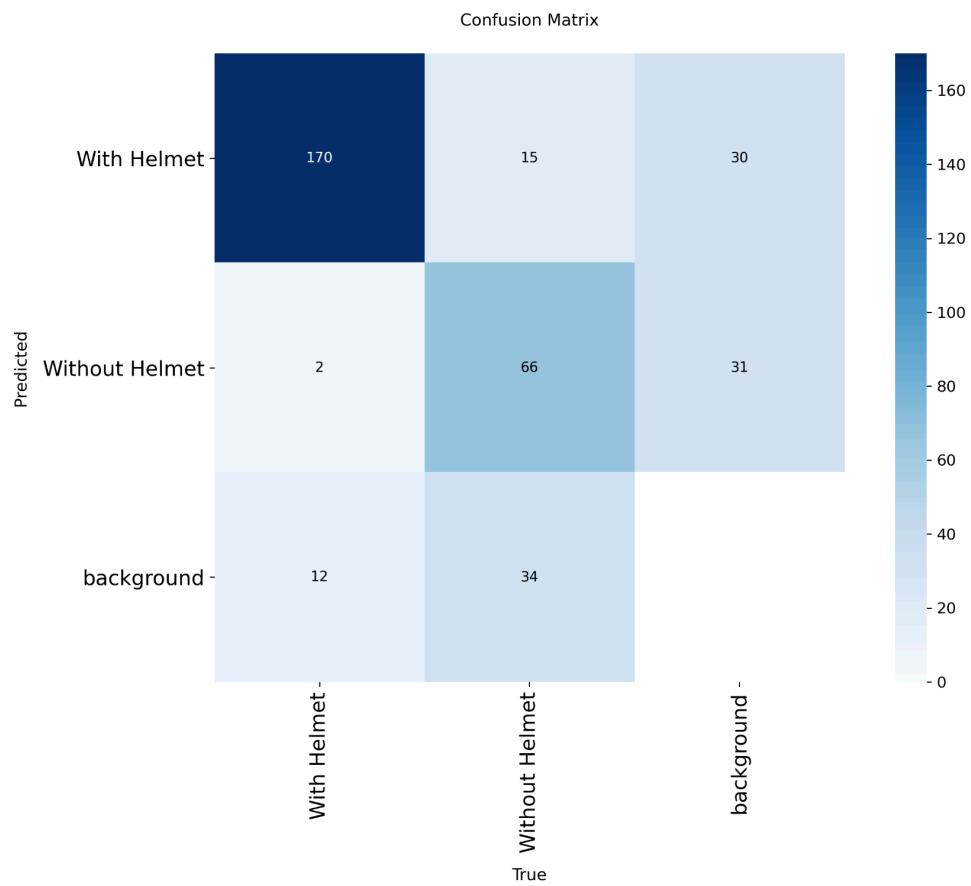
Sau khi thực hiện lệnh trên, hệ thống tiến hành huấn luyện mô hình trong 20 epochs và lưu kết quả tại thư mục: **runs/detect/helmet_debug/**

Thư mục này chứa các thông tin quan trọng phục vụ cho việc phân tích mô hình, bao gồm:

- File trọng số tốt nhất: best.pt
- File trọng số cuối cùng: last.pt
- Biểu đồ loss và metric: results.png



- Ma trận nhầm lẫn (confusion matrix)



- Các hình ảnh minh họa kết quả dự đoán



Kết quả huấn luyện:

Để kiểm chứng khách quan, em tiến hành chạy lại công cụ Validation trên trọng số tốt nhất (best.pt) vừa thu được

yolo detect val model=runs/detect/helmet_debug/weights/best.pt

data=dataset/data.yaml

Mô hình đạt được các chỉ số đánh giá trên tập validation như sau:

Chỉ số	Giá trị
Precision	0.758
Recall	0.738
mAP@0.5	0.758

mAP@0.5:0.95	0.334
--------------	-------

Kết quả theo từng lớp đối tượng:

Lớp	Precision	Recall	mAP@0.5	mAP@0.5:0.95
With Helmet	0.804	0.918	0.881	0.405
Without Helmet	0.711	0.557	0.634	0.262

Tốc độ xử lý trung bình 1 ảnh (ms):

preprocess	inference	postprocess	loss	Tổng
1.6	6.0	0.6	0.0	8.2

Từ kết quả trên, có thể nhận thấy:

- Mô hình đã có khả năng học được đặc trưng của dữ liệu chỉ sau 20 epoch.
- Lớp “With Helmet” đạt hiệu suất cao hơn đáng kể so với lớp “Without Helmet”, thể hiện qua chỉ số Recall và mAP.
 - Lớp "With Helmet": Đạt mAP@0.5 rất cao (0.881), cho thấy mô hình đã học rất tốt đặc trưng của các loại mũ bảo hiểm.
 - Lớp "Without Helmet": Chỉ số Recall còn thấp (0.557), phản ánh việc mô hình vẫn bị bỏ sót khoảng 44% các trường hợp không đội mũ bảo hiểm. Đây sẽ là trọng tâm cần cải thiện trong bước huấn luyện chính thức 50-100 epochs
- Tốc độ xử lý nhanh (8.2 ms mỗi khung hình), phù hợp với yêu cầu xử lý thời gian thực

Phân tích kết quả

Thông qua quá trình huấn luyện thử nghiệm, có thể rút ra một số nhận xét quan trọng:

- Mô hình học tốt: Chỉ số mAP@0.5 đạt mức trên 0.75 cho thấy mô hình đã học được các đặc trưng cơ bản của bài toán phát hiện mũ bảo hiểm.

- Chưa đạt hiệu suất tối ưu: Do số epoch còn thấp, mô hình chưa khai thác hết tiềm năng của dataset.
- Sự mất cân bằng giữa các lớp: Lớp “Without Helmet” có Recall thấp hơn, cho thấy mô hình còn gặp khó khăn trong việc phát hiện đầy đủ các đối tượng thuộc lớp này.
 - Recall thấp nghĩa là mô hình bị bỏ sót nhiều người không đội mũ. Điều này cực kỳ logic vì dữ liệu lớp này ít hơn và người không đội mũ thường khó nhận diện hơn (đầu người có thể lẫn với hậu cảnh hơn là cái mũ bảo hiểm to rõ)
- Không xuất hiện dấu hiệu overfitting rõ rệt: Do thời gian huấn luyện còn ngắn, mô hình chưa có dấu hiệu ghi nhớ dữ liệu.
- Với tổng thời gian xử lý chỉ 8.2 ms mỗi khung hình, mô hình đạt tốc độ xấp xỉ 122 FPS. Tốc độ này vượt xa tiêu chuẩn xử lý thời gian thực thông thường (khoảng 30-60 FPS).

Ý nghĩa của bước huấn luyện thử nghiệm

Bước huấn luyện thử nghiệm đóng vai trò quan trọng trong toàn bộ pipeline phát triển mô hình, cụ thể:

- Giúp xác nhận rằng mô hình có khả năng học từ dataset đã chuẩn bị.
- Cung cấp cơ sở để đánh giá và điều chỉnh các tham số huấn luyện.
- Là bước trung gian trước khi tiến hành huấn luyện chính thức với số epochs lớn hơn.

Kết quả đạt được từ bước này cho thấy mô hình và dataset đã sẵn sàng cho giai đoạn huấn luyện chính thức trong bước tiếp theo.

2.3. Final Training - 100 epochs (Huấn luyện chính thức)

Sau khi hoàn thành bước huấn luyện thử nghiệm và xác nhận rằng mô hình có khả năng học tốt từ dữ liệu, bước tiếp theo là tiến hành huấn luyện chính thức (final training) với số lượng epoch lớn hơn nhằm tối ưu hiệu suất của mô hình.

Mục tiêu của giai đoạn này bao gồm:

- Tối đa hóa độ chính xác của mô hình trên tập dữ liệu
- Cải thiện khả năng phát hiện đối tượng, đặc biệt là lớp “Without Helmet”

- Giảm thiểu sai số và hiện tượng bỏ sót đối tượng
- Tìm ra điểm hội tụ tối ưu của mô hình thông qua cơ chế early stopping

Trong bước này, mô hình YOLOv8n pretrained tiếp tục được fine-tune trên dataset với cấu hình huấn luyện được thiết lập như sau:

Lệnh thực hiện:

```
yolo detect train data=dataset/data.yaml model=yolov8n.pt epochs=100  
imgsz=416 batch=8 patience=50 name=helmet_final
```

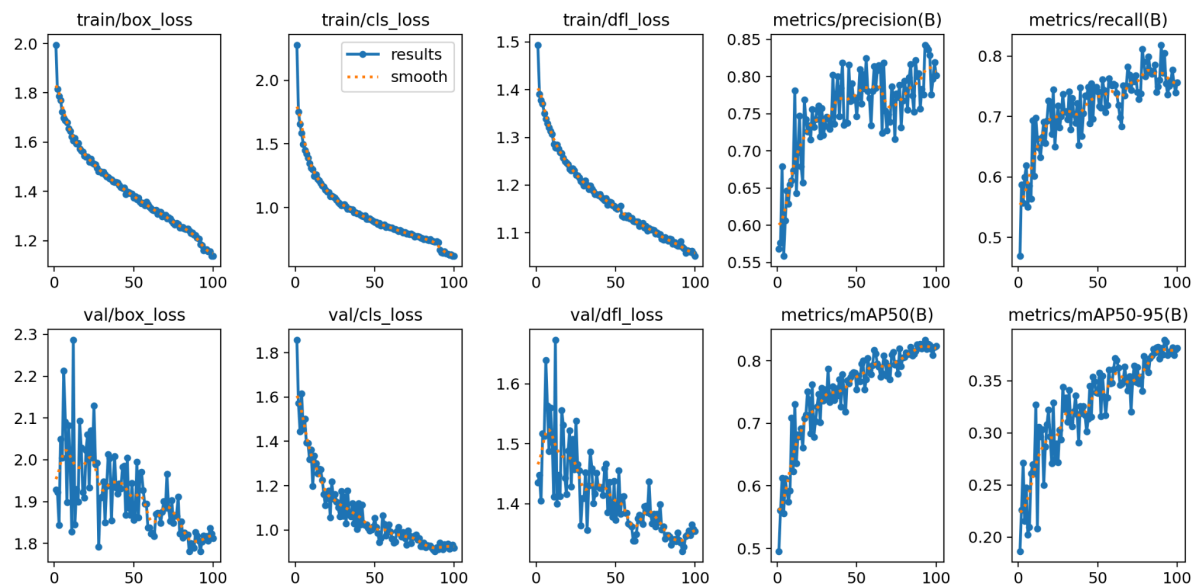
Giải thích các tham số:

- **data=dataset/data.yaml:** chỉ định file cấu hình dataset, bao gồm đường dẫn tới các tập train, validation và test
- **model=yolov8n.pt:** sử dụng mô hình YOLOv8 phiên bản nano đã được pretrained
- **epochs=100:** số epoch huấn luyện chính thức
- **imgsz=416:** kích thước ảnh đầu vào, phù hợp với dataset đã được resize trước đó
- **batch=8:** số lượng ảnh được xử lý trong mỗi batch, phù hợp với tài nguyên GPU hiện có
- **patience=50:** nếu trong 50 epochs liên tiếp không có cải thiện về metric, quá trình huấn luyện sẽ tự động dừng (early stopping)
- **name=helmet_final:** tên thư mục lưu kết quả huấn luyện

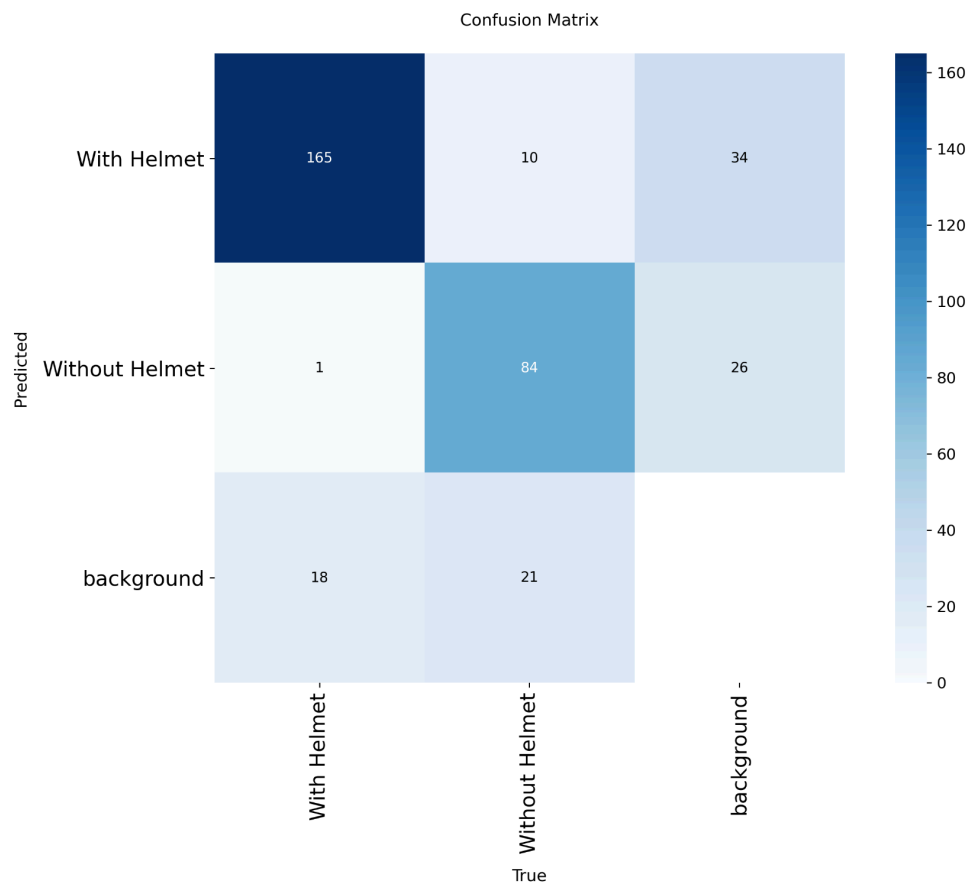
Sau khi thực hiện lệnh trên, hệ thống tiến hành huấn luyện mô hình trong 100 epochs và lưu kết quả tại thư mục: **runs/detect/helmet_final/**

Thư mục này chứa các thông tin quan trọng phục vụ cho việc phân tích mô hình, bao gồm:

- File trọng số tốt nhất: best.pt
- File trọng số cuối cùng: last.pt
- Biểu đồ loss và metric: results.png



- Ma trận nhầm lẫn (confusion matrix)



- Các hình ảnh minh họa kết quả dự đoán



Kết quả huấn luyện:

Để kiểm chứng khách quan, em tiến hành chạy lại công cụ Validation trên trọng số tốt nhất (best.pt) vừa thu được

yolo detect val model=runs/detect/helmet_final/weights/best.pt

data=dataset/data.yaml

Mô hình đạt được các chỉ số đánh giá trên tập validation như sau:

Chỉ số	Giá trị
Precision	0.776
Recall	0.805
mAP@0.5	0.826

mAP@0.5:0.95	0.389
--------------	-------

Kết quả theo từng lớp đối tượng:

Lớp	Precision	Recall	mAP@0.5	mAP@0.5:0.95
With Helmet	0.795	0.897	0.896	0.45
Without Helmet	0.757	0.713	0.757	0.328

Tốc độ xử lý trung bình 1 ảnh (ms):

preprocess	inference	postprocess	loss	Tổng
1.7	7.1	0.7	0.0	9.5

So sánh với huấn luyện thử nghiệm:

Chỉ số	20 epochs	100 epochs
Precision	0.758	0.776
Recall	0.738	0.805
mAP@0.5	0.758	0.826
mAP@0.5:0.95	0.334	0.389

Việc tăng số lượng epoch đã giúp mô hình cải thiện đáng kể hiệu suất. Điều này cho thấy rằng mô hình vẫn còn khả năng học thêm sau 20 epoch và việc huấn luyện dài hơn là cần thiết để đạt được kết quả tối ưu.

Phân tích kết quả:

Thông qua quá trình huấn luyện chính thức với 100 epoch, có thể rút ra một số nhận xét quan trọng về hiệu suất của mô hình như sau:

- Mô hình đạt hiệu suất cao:
 - Chỉ số mAP@0.5 đạt mức khoảng 0.826 cho thấy mô hình đã học tốt các đặc trưng của bài toán phát hiện mũ bảo hiểm. So với giai đoạn huấn luyện thử nghiệm (0.758), hiệu suất đã được cải thiện

rõ rệt, chứng tỏ việc tăng số lượng epoch giúp mô hình khai thác tốt hơn thông tin từ dataset.

- Khả năng phát hiện đối tượng được cải thiện đáng kể:
 - Giá trị Recall tăng lên khoảng 0.805 cho thấy mô hình đã giảm được tình trạng bỏ sót đối tượng so với trước. Điều này đặc biệt quan trọng trong các bài toán giám sát an toàn, nơi việc bỏ sót người không đội mũ bảo hiểm có thể gây ảnh hưởng lớn đến hệ thống cảnh báo.
- Hiệu suất giữa các lớp trở nên cân bằng hơn:
 - So với giai đoạn huấn luyện 20 epoch, lớp “Without Helmet” đã có sự cải thiện rõ rệt về Recall và mAP. Tuy vẫn thấp hơn lớp “With Helmet”, nhưng khoảng cách giữa hai lớp đã được thu hẹp đáng kể.
 - Điều này cho thấy mô hình đã học tốt hơn các đặc trưng của lớp khó, dù dữ liệu vẫn có sự mất cân bằng.
- Hiện tượng bỏ sót vẫn còn tồn tại nhưng đã giảm:
 - Recall của lớp “Without Helmet” đạt khoảng 0.713, cho thấy mô hình vẫn còn bỏ sót một số trường hợp. Đây là điều hợp lý do:
 - Số lượng dữ liệu của lớp này ít hơn
 - Đặc trưng của “không đội mũ” khó phân biệt hơn so với “có đội mũ”
 - Đầu người không đội mũ dễ bị nhầm lẫn với background
 - Dựa trên Confusion Matrix, ta thấy mô hình chủ yếu nhầm lẫn giữa lớp “Without Helmet” và “Background”. Điều này xác nhận giả thuyết rằng đầu người không đội mũ bảo hiểm trong môi trường giao thông phức tạp dễ bị lẫn vào bối cảnh phía sau. Tuy nhiên, tỷ lệ nhầm lẫn giữa “With Helmet” và “Without Helmet” là rất thấp, cho thấy mô hình phân loại giữa hai trạng thái này khá chính xác
- Không xuất hiện dấu hiệu Overfitting rõ rệt:
 - Biểu đồ kết quả (results.png) cho thấy đường cong Loss trên cả tập train và validation đều giảm ổn định, trong khi các chỉ số

mAP tăng đều và hội tụ tốt, khẳng định mô hình đang ở trạng thái tốt, không bị hiện tượng Overfitting

- Mô hình được thiết lập cơ chế early stopping với patience=50 như một phương án bảo vệ. Tuy nhiên, kết quả thực tế cho thấy mô hình liên tục cải thiện và đạt điểm tối ưu ở những epoch cuối (epoch 93), do đó quá trình huấn luyện đã hoàn thành trọn vẹn 100 epoch để thu được bộ trọng số tốt nhất
- Hiệu suất xử lý thời gian thực rất cao:
 - Với tổng thời gian xử lý khoảng 9.5 ms mỗi ảnh (bao gồm preprocess, inference và postprocess), mô hình đạt tốc độ xấp xỉ 105 FPS.
 - Tốc độ này vượt xa yêu cầu của các hệ thống thời gian thực (thường khoảng 30–60 FPS), cho thấy mô hình hoàn toàn có thể được triển khai trong các ứng dụng giám sát giao thông hoặc camera an ninh.

Kết luận:

Giai đoạn huấn luyện chính thức đã giúp xây dựng được một mô hình phát hiện mũ bảo hiểm với độ chính xác cao. Các chỉ số đánh giá cho thấy mô hình có khả năng phát hiện đối tượng tốt trong nhiều điều kiện khác nhau, đồng thời giảm thiểu được sai số so với giai đoạn trước.

Kết quả này là cơ sở quan trọng để triển khai mô hình vào các bài toán thực tế trong các giai đoạn tiếp theo của dự án.

3. Kết quả đạt được trong tuần 3

Sau khi hoàn thành các công việc trong tuần thứ ba, các kết quả chính đạt được bao gồm:

- Thực hiện thành công bước sanity check, qua đó xác nhận pipeline huấn luyện của mô hình YOLOv8 hoạt động ổn định và dataset có thể được đọc đúng định dạng.

- Tiến hành huấn luyện thử nghiệm (debug training) với 20 epochs, giúp đánh giá khả năng học của mô hình và quan sát xu hướng thay đổi của các chỉ số như loss, Precision, Recall và mAP.
 - Xác nhận rằng mô hình có khả năng học tốt từ dataset, thể hiện qua việc các giá trị loss giảm dần và mAP tăng lên trong quá trình huấn luyện.
 - Phân tích được sự khác biệt hiệu suất giữa hai lớp đối tượng “With Helmet” và “Without Helmet”, từ đó nhận diện được ảnh hưởng của sự mất cân bằng dữ liệu đến kết quả mô hình.
- Thực hiện huấn luyện chính thức (final training) với 100 epochs, giúp mô hình đạt được hiệu suất cao hơn so với giai đoạn thử nghiệm.
 - Đánh giá mô hình bằng tập validation thông qua các chỉ số như Precision, Recall, mAP@0.5 và mAP@0.5:0.95, đảm bảo kết quả có tính khách quan và đáng tin cậy.
 - So sánh kết quả giữa các giai đoạn huấn luyện (20 epochs và 100 epochs), từ đó chứng minh rằng việc tăng số epoch giúp cải thiện đáng kể hiệu suất của mô hình.
 - Xác nhận rằng mô hình đạt được tốc độ xử lý cao (trên 100 FPS), đáp ứng yêu cầu của các hệ thống xử lý thời gian thực.
 - Thu được mô hình tối ưu (best.pt) có thể sử dụng cho các bước triển khai hoặc thử nghiệm trong môi trường thực tế.

Những kết quả đạt được trong tuần thứ ba cho thấy mô hình đã được huấn luyện thành công và đạt hiệu suất tốt, tạo nền tảng quan trọng cho các bước tiếp theo như tối ưu mô hình, triển khai thực tế hoặc tích hợp vào hệ thống ứng dụng.

4. Khó khăn gặp phải và cách khắc phục

Trong quá trình thực hiện tuần thứ ba của dự án, một số khó khăn đã phát sinh liên quan đến việc huấn luyện và đánh giá mô hình YOLOv8. Các vấn đề này chủ yếu xuất phát từ việc đảm bảo mô hình học đúng từ dữ liệu, lựa chọn cấu hình huấn luyện phù hợp, cũng như phân tích kết quả một cách chính xác.

4.1. Khó khăn trong việc kiểm tra tính đúng đắn của pipeline huấn luyện

Một trong những khó khăn ban đầu trong tuần này là việc đảm bảo pipeline huấn luyện hoạt động chính xác trước khi tiến hành train chính thức. Nếu pipeline gặp lỗi (ví dụ: sai format label, sai đường dẫn, lỗi đọc dữ liệu), toàn bộ quá trình huấn luyện dài có thể bị lãng phí.

Ban đầu, việc chỉ chạy predict bằng mô hình pretrained ở tuần trước chưa đủ để đảm bảo rằng toàn bộ pipeline huấn luyện (training pipeline) hoạt động đúng, vì bước predict không sử dụng label.

Để khắc phục vấn đề này, em đã thực hiện bước Sanity Check bằng cách huấn luyện mô hình với số epoch rất nhỏ (2 epoch). Thông qua bước này, em có thể kiểm tra:

- Dataset có được load đúng hay không
- Label có bị lỗi hay không
- Loss có giảm hay không
- Mô hình có học được hay không

Kết quả sanity check cho thấy loss giảm và các chỉ số mAP đạt giá trị hợp lý, từ đó xác nhận pipeline huấn luyện hoạt động ổn định và có thể tiếp tục sang các bước tiếp theo.

4.2. Khó khăn trong việc lựa chọn số lượng epoch phù hợp

Việc lựa chọn số lượng epoch phù hợp là một vấn đề quan trọng trong quá trình huấn luyện mô hình. Nếu số epoch quá ít, mô hình sẽ chưa học đủ và cho kết quả kém. Ngược lại, nếu số epoch quá lớn, mô hình có thể bị overfitting, làm giảm khả năng tổng quát hóa trên dữ liệu mới.

Ban đầu, em chưa xác định được số epoch tối ưu cho bài toán, do chưa có kinh nghiệm thực nghiệm với dataset hiện tại.

Để giải quyết vấn đề này, em đã áp dụng chiến lược huấn luyện theo từng giai đoạn:

- Train nhanh với 2 epoch (Sanity Check) để kiểm tra pipeline
- Train thử nghiệm với 20 epoch (Debug) để quan sát xu hướng học
- Train chính thức với 100 epoch kết hợp với cơ chế Early Stopping của YOLOv8

Cơ chế Early Stopping cho phép mô hình tự động dừng huấn luyện khi không còn cải thiện sau một số epochs nhất định, giúp tránh lãng phí thời gian và hạn chế overfitting. Tuy nhiên, trên thực tế mô hình đã hoàn thành trọn vẹn 100 epoch mà không kích hoạt dừng sớm, do các chỉ số vẫn tiếp tục được cải thiện và đạt điểm tối ưu tại epoch thứ 93

Nhờ đó, mô hình có thể đạt được hiệu suất tốt nhất mà không cần phải xác định chính xác số epoch ngay từ đầu.

4.3. Khó khăn do sự mất cân bằng dữ liệu giữa các lớp

Dataset sử dụng trong bài toán có sự mất cân bằng giữa hai lớp đối tượng:

- Lớp “With Helmet” chiếm tỷ lệ lớn hơn
- Lớp “Without Helmet” có số lượng ít hơn

Trong quá trình huấn luyện, điều này dẫn đến việc mô hình học tốt hơn đối với lớp “With Helmet”, trong khi hiệu suất đối với lớp “Without Helmet” thấp hơn, đặc biệt là về chỉ số Recall.

Kết quả thực nghiệm cho thấy mô hình có xu hướng bỏ sót một số đối tượng không đội mũ bảo hiểm. Điều này là hợp lý vì:

- Số lượng dữ liệu của lớp này ít hơn
- Đặc trưng nhận diện khó hơn (đầu người không đội mũ dễ bị nhầm với background)

Để khắc phục vấn đề này, em đã:

- Theo dõi riêng từng lớp trong quá trình đánh giá mô hình
- Phân tích confusion matrix để xác định lỗi của mô hình

Trong các bước tiếp theo, có thể cải thiện thêm bằng cách:

- Thu thập thêm dữ liệu cho lớp “Without Helmet”
- Áp dụng kỹ thuật cân bằng dữ liệu (class balancing)

4.4. Khó khăn trong việc lựa chọn batch size phù hợp với GPU

Trong quá trình huấn luyện, việc lựa chọn batch size phù hợp với phần cứng là một vấn đề quan trọng. GPU sử dụng trong dự án có dung lượng bộ nhớ 4GB, do đó không thể sử dụng batch size quá lớn.

Ban đầu, em cân nhắc tăng batch size từ 8 lên 16 để cải thiện độ ổn định của gradient và có thể giúp mô hình học tốt hơn. Tuy nhiên, việc này có thể dẫn đến:

- Lỗi tràn bộ nhớ GPU (Out of Memory)
- Không cải thiện đáng kể độ chính xác trong một số trường hợp

Sau khi thử nghiệm và đánh giá, em quyết định sử dụng batch size = 8 vì:

- Phù hợp với giới hạn bộ nhớ GPU
- Đảm bảo quá trình huấn luyện ổn định
- Độ chính xác đạt được ở mức tốt

4.5. Khó khăn trong việc phân tích và đánh giá mô hình

Một khó khăn khác trong tuần này là việc hiểu và phân tích các chỉ số đánh giá của mô hình, bao gồm:

- Precision
- Recall
- mAP@0.5
- mAP@0.5:0.95

Ban đầu, việc chỉ nhìn vào một chỉ số (ví dụ mAP) có thể gây hiểu nhầm về hiệu suất thực tế của mô hình.

Để khắc phục vấn đề này, em đã:

- Kết hợp nhiều chỉ số để đánh giá toàn diện
- Phân tích kết quả theo từng lớp đối tượng
- Sử dụng confusion matrix để hiểu rõ các lỗi dự đoán
- Quan sát thêm các biểu đồ loss và kết quả predict

Nhờ đó, việc đánh giá mô hình trở nên chính xác và có cơ sở hơn, giúp đưa ra các quyết định cải thiện hợp lý.

5. Kế hoạch thực hiện trong tuần tiếp theo

Sau khi đã hoàn thành quá trình huấn luyện và đánh giá mô hình trong tuần thứ ba, tuần tiếp theo của dự án sẽ tập trung vào việc cải thiện hiệu suất mô hình và triển khai ứng dụng thực tế. Các công việc này nhằm nâng cao độ chính xác, khả năng tổng quát hóa cũng như khả năng ứng dụng của hệ thống phát hiện mũ bảo hiểm.

5.1. Tối ưu hóa mô hình

Mặc dù mô hình đã đạt được kết quả tương đối tốt sau quá trình huấn luyện chính thức, vẫn còn một số điểm cần được cải thiện, đặc biệt là đối với lớp “Without Helmet”. Do đó, trong tuần tiếp theo, em sẽ tiến hành tối ưu hóa mô hình thông qua các hướng sau:

- Điều chỉnh các siêu tham số (hyperparameters) như:
 - learning rate
 - batch size
 - số epoch
- Thử nghiệm với các phiên bản mô hình khác như YOLOv8s hoặc YOLOv8m để so sánh hiệu suất.
- Tăng cường dữ liệu (data augmentation) nhằm cải thiện khả năng nhận diện trong các điều kiện phức tạp.
- Cân bằng lại dữ liệu giữa các lớp nếu cần thiết, đặc biệt là lớp “Without Helmet”.

Việc tối ưu hóa này giúp mô hình đạt được hiệu suất cao hơn và giảm thiểu các lỗi nhận diện.

5.2. Đánh giá mô hình trên tập dữ liệu test

Trong tuần thứ ba, mô hình chủ yếu được đánh giá trên tập validation. Do đó, trong tuần tiếp theo, em sẽ tiến hành đánh giá mô hình trên tập test để có cái nhìn khách quan hơn về khả năng tổng quát hóa.

Các bước thực hiện bao gồm:

- Sử dụng mô hình tốt nhất (best.pt) để chạy trên tập test
- Thu thập các chỉ số đánh giá:
 - Precision
 - Recall
 - mAP@0.5
 - mAP@0.5:0.95
- Phân tích confusion matrix để đánh giá chi tiết các lỗi của mô hình

Việc đánh giá trên tập test giúp đảm bảo mô hình không chỉ hoạt động tốt trên dữ liệu đã thấy mà còn có khả năng áp dụng trong thực tế.

